Cloud Computing Lab 5 – Assignment 2

Implementing Amazon DynamoDB (NoSQL) as the Opposite Database Type for the Microblog Application

Application	Microblog (Flask)
Database	Amazon DynamoDB (NoSQL)
Table Name	MicroblogPosts
Partition Key	id (String / HASH)
EC2 Application URL	http://3.111.37.11
AWS Region	ap-south-1 (Asia Pacific – Mumbai)
GitHub Repository	github.com/appisal/aws-cloud-lab5-microblog

1. Architecture

For Assignment 2, the opposite database type to Lab 4 (RDS/SQL) was implemented: Amazon DynamoDB, a fully managed NoSQL service. The Microblog EC2 instance was granted a dedicated IAM role (**Microblog-DynamoDB-EC2-Role**) rather than hardcoded AWS credentials, allowing the application and AWS CLI on the instance to call the DynamoDB API directly.

```
Internet → EC2 (Microblog App, IAM Role) → DynamoDB API → MicroblogPosts Table
```

2. DynamoDB Table Configuration

Property	Value
Table Name	MicroblogPosts
AWS Region	ap-south-1
Table Status	ACTIVE
Partition Key	id
Key Type	HASH (String)

3. Security Configuration

Access to DynamoDB is granted exclusively through the IAM role **Microblog-DynamoDB-EC2-Role**, attached to the EC2 instance (i-0afd3d12c5d858ec4). No AWS access keys are hardcoded in the application or on the instance. The attached identity was verified using `aws sts get-caller-identity`, which confirmed the caller as the assumed role `Microblog-DynamoDB-EC2-Role`.

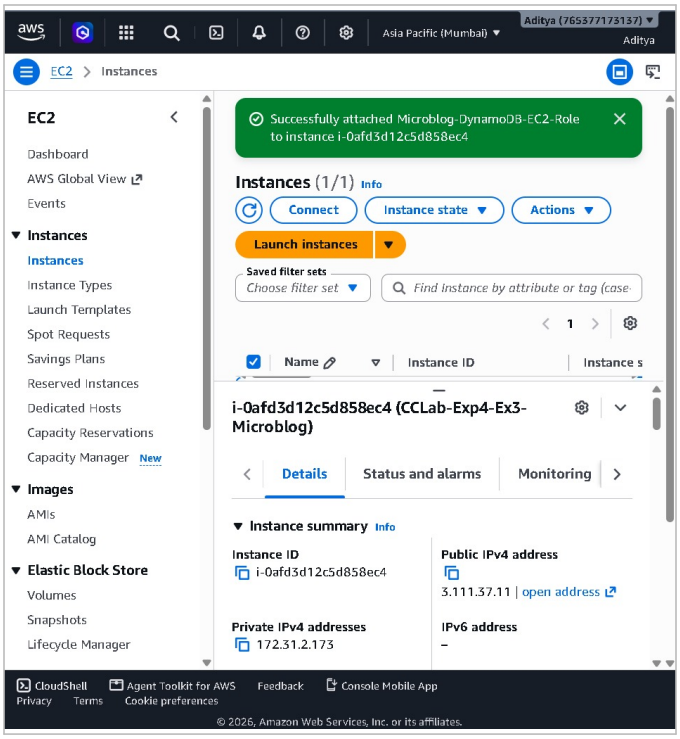

Figure 1: EC2 console – Microblog-DynamoDB-EC2-Role successfully attached to instance i-0afd3d12c5d858ec4

```
Microblog-DynamoDB-EC2-Roleubuntu@ip-172-31-2-173:/tmp$ aws sts get-caller-i
aws sts get-caller-identity
{
    "UserId": "AROA3ENAI22IXMXUT57R4:i-0afd3d12c5d858ec4",
    "Account": "765377173137",
    "Arn": "arn:aws:sts::765377173137:assumed-role/Microblog-DynamoDB-EC2-Ro
le/i-0afd3d12c5d858ec4"
}
ubuntu@ip-172-31-2-173:/tmp$
```

Figure 2: aws sts get-caller-identity run on the EC2 instance – confirms the assumed role identity

4. DynamoDB Attribute Types (5 required, 6 demonstrated)

Attribute	DynamoDB Type	Example
id	String (S)	"1"
username	String (S)	"aditya"
age	Number (N)	21 → 22 (after update)
active	Boolean (BOOL)	true → false (after update)
tags	List (L)	["cloud", "aws", "dynamodb"]
profile	Map (M)	{"city": "Mumbai", "course": "Computer Engineering"}

```
ubuntu@ip-172-31-2-173:/tmp$ aws dynamodb describe-table --table-name Microb
logPosts --region ap-south-1 --query "Table.[TableName,TableStatus,TableArn]
"
[
    "MicroblogPosts",
    "ACTIVE",
    "arn:aws:dynamodb:ap-south-1:765377173137:table/MicroblogPosts"
]
ubuntu@ip-172-31-2-173:/tmp$
```

Figure 3: aws dynamodb describe-table – MicroblogPosts confirmed ACTIVE with its table ARN

```
ubuntu@ip-172-31-2-173:/tmp$ aws dynamodb put-item \
  --table-name MicroblogPosts \
  --region ap-south-1 \
  --item '{
    "id": {"S": "1"},
    "username": {"S": "aditya"},
    "age": {"N": "21"},
    "active": {"BOOL": true},
    "tags": {"L": [{"S": "cloud"}, {"S": "aws"}, {"S": "dynamodb"}]},
    "profile": {"M": {
      "city": {"S": "Mumbai"},
      "course": {"S": "Computer Engineering"}
    }},
    "created_at": {"S": "2026-08-19"}
  }'
ubuntu@ip-172-31-2-173:/tmp$ aws dynamodb get-item \
  --table-name MicroblogPosts \
  --region ap-south-1 \
  --key '{"id":{"S":"1"}}'
{
    "Item": {
        "active": {
            "BOOL": true
        },
        "created_at": {
            "S": "2026-08-19"
        },
        "profile": {
            "M": {
                "city": {
                    "S": "Mumbai"
                },
                "course": {
                    "S": "Computer Engineering"
                }
            }
        },
        "username": {
            "S": "aditya"
        },
        "id": {
            "S": "1"
        },
        "tags": {
            "L": [
```

Figure 4: put-item (Create) and get-item (Read) – item id=1 written and retrieved with all attribute types

```
        },
        "username": {
            "S": "aditya"
        },
        "id": {
            "S": "1"
        },
        "tags": {
            "L": [
                {
                    "S": "cloud"
                },
                {
                    "S": "aws"
                },
                {
                    "S": "dynamodb"
                }
            ]
        },
        "age": {
            "N": "21"
        }
    }
}
ubuntu@ip-172-31-2-173:/tmp$
```

Figure 5: get-item output continued – username, tags (List), and age (Number) attributes visible

5. CRUD Operations – AWS CLI Level

Operation	Command	Result
Create	put-item	Item id=1 created with all 6 attributes
Read	get-item	Item retrieved successfully via partition key
Update	update-item	age: 21→22, active: true→false, confirmed via ALL_NEW
Delete	delete-item	Item removed, confirmed via ALL_OLD; scan afterward returned Count: 0

```
ubuntu@ip-172-31-2-173:/tmp$ aws dynamodb update-item \
  --table-name MicroblogPosts \
  --region ap-south-1 \
  --key '{"id":{"S":"1"}}' \
  --update-expression "SET age = :age, active = :active" \
  --expression-attribute-values '{":age":{"N":"22"},":active":{"BOOL":false}
}' \
  --return-values ALL_NEW \
  --output json
{
    "Attributes": {
        "active": {
            "BOOL": false
        },
        "username": {
            "S": "aditya"
        },
        "id": {
            "S": "1"
        },
        "tags": {
            "L": [
                {
                    "S": "cloud"
                },
                {
                    "S": "aws"
                },
                {
                    "S": "dynamodb"
                }
            ]
        },
        "age": {
            "N": "22"
        },
        "profile": {
            "M": {
                "city": {
                    "S": "Mumbai"
                },
                "course": {
                    "S": "Computer Engineering"
                }
            }
```

Figure 6: update-item – age changed 21→22 and active changed true→false, returned via ALL_NEW

```
ubuntu@ip-172-31-2-173:/tmp$ aws dynamodb delete-item \
  --table-name MicroblogPosts \
  --region ap-south-1 \
  --key '{"id":{"S":"1"}}' \
  --return-values ALL_OLD \
  --output json
{
    "Attributes": {
        "active": {
            "BOOL": false
        },
        "username": {
            "S": "aditya"
        },
        "id": {
            "S": "1"
        },
        "tags": {
            "L": [
                {
                    "S": "cloud"
                },
                {
                    "S": "aws"
                },
                {
                    "S": "dynamodb"
                }
            ]
        },
        "age": {
            "N": "22"
        },
        "profile": {
            "M": {
                "city": {
                    "S": "Mumbai"
                },
                "course": {
                    "S": "Computer Engineering"
                }
            }
        }
    }
}
```

Figure 7: delete-item – item id=1 deleted, full deleted record returned via ALL_OLD

```
ubuntu@ip-172-31-2-173:/tmp$ aws dynamodb scan \
  --table-name MicroblogPosts \
  --region ap-south-1 \
  --output json
{
    "Items": [],
    "Count": 0,
    "ScannedCount": 0,
    "ConsumedCapacity": null
}
ubuntu@ip-172-31-2-173:/tmp$
```

Figure 8: aws dynamodb scan after deletion – Count: 0, confirming the item no longer exists

6. Application-Level CRUD Demonstration

The Microblog application running on EC2 was also tested end-to-end through its web interface, logged in as user aditya2:

Operation	Application Action	Result Observed
Create	Submitted a new post	“Your post is now live!”
Read	Feed loaded after posting	Post displayed under aditya2's profile
Update	Edited the post text	“Your post has been updated.”
Delete	Deleted the post (confirm dialog accepted)	Post removed from feed

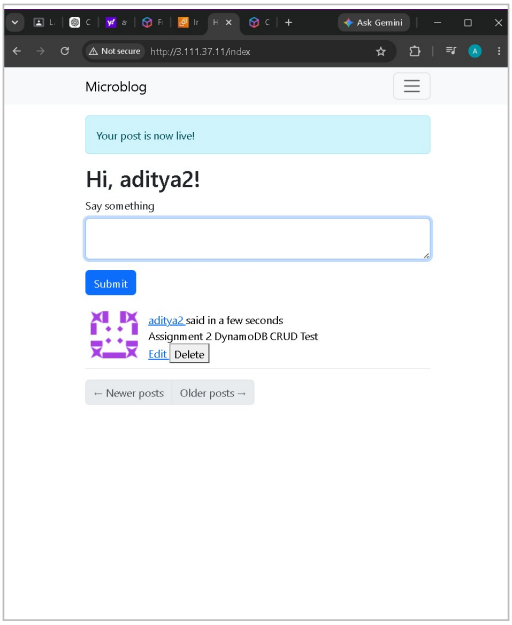

Figure 9: Create – user aditya2 submits a new post; “Your post is now live!” confirmation shown

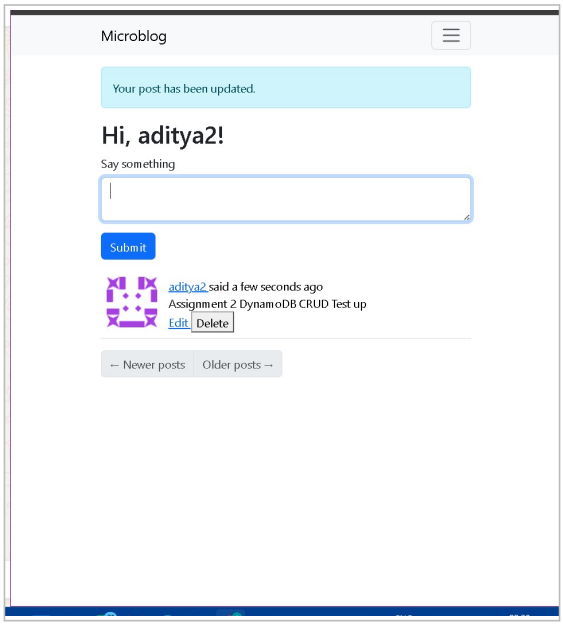

Figure 10: Update – post text edited; “Your post has been updated.” confirmation shown

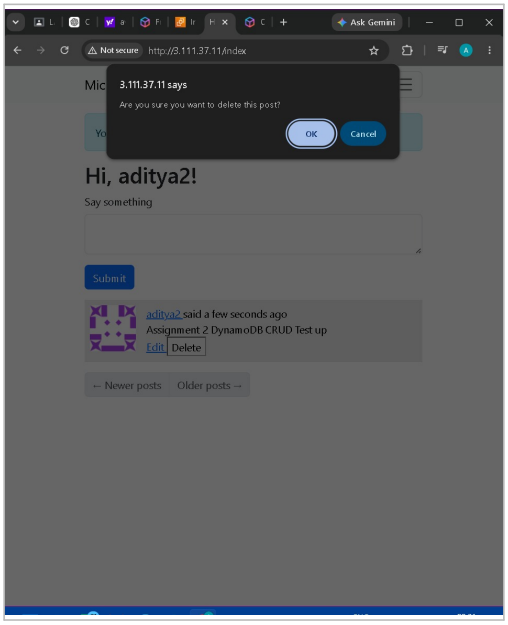

Figure 11: Delete – browser confirmation dialog accepted before the post is removed

7. Evidence Submitted

- EC2 instance detail showing IAM role Microblog-DynamoDB-EC2-Role attached
- aws sts get-caller-identity output confirming the assumed role
- aws dynamodb describe-table output showing table status ACTIVE
- put-item / get-item output showing all 6 attributes and 5+ datatypes
- update-item output (ALL_NEW) showing changed values
- delete-item output (ALL_OLD) followed by scan confirming Count: 0
- Application screenshots: post created, post updated, delete confirmation dialog

8. Deliverables

- GitHub Repository: github.com/appisal/aws-cloud-lab5-microblog
- EC2 Application URL: http://3.111.37.11
- DynamoDB Table: MicroblogPosts (Region: ap-south-1)
- CRUD: Create, Read, Update, Delete – demonstrated via both AWS CLI and the live application

- Datatype coverage: String, Number, Boolean, List, Map (5 required, all present)
- Security: DynamoDB access via EC2 IAM Role only – no hardcoded AWS keys